

Compiladores, 2025/2026

Aula prática 6

Compilador de uma linguagem aritmética de inteiros

Fernando Lobo

1 Introdução

Na aula teórica 13 foi ilustrado a implementação de um compilador para uma *calculator language* muito simples. O compilador traduz um programa feito nesta linguagem para bytecodes de uma máquina virtual. Para além do compilador, também foi ilustrado a implementação de uma máquina virtual capaz de correr os bytecodes gerados.

O código feito e ilustrado pelo docente está disponível na tutoria na pasta Aulas PL/p6, através do ficheiro compactado `calcCompiler.zip`

2 Objectivo

O objectivo desta aula é estudar muito bem o código que lhe foi disponibilizado.

Comece por criar um projecto com o IntelliJ, configure o ANTLR, e faça *Generate ANTLR Recognizer* para o ANTLR gerar o código apropriado (Lexer, Parser, etc) para a gramática `Calc.g4`

Depois certifique-se que o código compila e executa. Note que o código tem 2 programas: `calCompiler` e `calVM`. Crie ficheiros de configuração para correr ambos os programas. Note que o `calCompiler` permite a opção `-asm` que faz com que o compilador apresente no ecrã o código gerado em formato de texto (*assembly*). A máquina virtual `calVM`, por sua vez, permite a opção `-trace` que mostra a execução da máquina virtual, passo a passo.

3 Linguagem fonte

A language, implementada apenas permite operações sobre números inteiros. Tem apenas os 4 operadores aritméticos binários $+$, $-$, $*$, $/$, o operador unário $-$, e os parêntesis $($ e $)$ para agrupar sub-expressões.

Um input sintaticamente válido nesta linguagem é uma sequência de 1 ou mais instruções terminadas por um caracter de mudança de linha. Cada instrução tem a palavra reservada `print` seguido de uma expressão aritmética.

Exemplo de input válido:

```
print 5+7
print -2+3*4-(31-30)
```

4 Instruções da máquina virtual

A máquina virtual é *stack-based*. Isto é, não usa registos para armazenar resultados temporários. Todos os operandos e resultados temporários são guardados no *runtime stack*.

A máquina virtual tem as seguintes instruções:

Nome	OpCode	Argumento	Descrição
iconst	0	inteiro n	faz $push(n)$
iuminus	1		faz $x = pop()$, seguido de $push(-x)$
iadd	2		faz $right = pop()$, seguido de $left = pop()$, seguido de $push(left + right)$
isub	3		faz $right = pop()$, seguido de $left = pop()$, seguido de $push(left - right)$
imult	4		faz $right = pop()$, seguido de $left = pop()$, seguido de $push(left * right)$
idiv	5		faz $right = pop()$, seguido de $left = pop()$, seguido de $push(left / right)$
iprint	6		faz $x = pop()$, e depois escreve o valor de x no ecrã

5 Ler e escrever bytes para um ficheiro

O Java tem mecanismos para se poder escrever um valor de um certo tipo como uma sequência de bytes, e também para fazer o seu reverso. Isto é, ler uma sequência de bytes e interpretá-lo como um valor de um certo tipo.

Isso pode ser feito usando as classes *DataInputStream* e *DataOutputStream*, fornecidas pelo Java.

6 Ideia para explorar

Tente acrescentar o operador de potência à linguagem, tal como já fizemos numa aula prática passada, e altere o compilador de modo a gerar código apropriado. Para tal pode acrescentar uma nova instrução à máquina virtual, *ipower*, que funciona de modo análogo a *iadd*, *isub*, *imult*, e *idiv*.